

INFO-Y115 - Secure software design and web security

MediVault - Secure Medical Records Platform



Project Team

Bosschem Nicolas - 000575046

Bui The - 000546997

Fernandez Ojeda Franklin - 000541971

Herbiet Dorian - 000513131

Otto Aleksandra - 000569128

Rafaat Moheeb Eskandar Abanoub - 000567614

Contents

1	Introduction	3
1.1	Project Context	3
1.2	System Overview	3
1.3	Security Philosophy	3
2	Threat Model	4
3	Patient Authentication and Device Binding	4
3.1	Motivation and Security Requirements	4
3.2	Why WebAuthn?	4
3.3	WebAuthn Implementation Details	5
3.4	UMK and PMK Design with PRF Support	5
3.4.1	Primary Master Key (PMK)	5
3.4.2	User Master Key (UMK) Generation and Storage	5
4	End-to-End Encryption Scheme	6
4.1	File Encryption Architecture	6
4.2	Doctor Access and Key Sharing	6
4.3	Access Revocation Mechanism	6
5	Doctor Authentication and Key Management	7
5.1	Certificate-Based Authentication	7
5.2	Password-Based Private Key Protection	7
5.3	WebAuthn Enhancement (Future Implementation)	8
6	PKI Implementation and Security Choices	8
6.1	Hierarchical Certificate Authority Structure	8
6.2	Certificate Lifecycle Management	9
7	Server Trust Configuration	10
8	Secure Input Processing and Validation	10
8.1	Context-Specific Security Requirements	10
8.2	Input Validation Strategy	10
8.3	XSS Protection	11
8.4	Canonicalization and Normalization	11
8.5	Denial of Service Prevention	11
8.6	Defensive Layered Architecture	11
8.7	STRIDE Threat Coverage	12
9	Software Architecture and Design Decisions	12
9.1	Use of Data Transfer Objects (DTOs)	12
9.1.1	Motivation for DTOs:	12
9.1.2	Example:	12
9.2	Layered Architecture	13
9.3	API Design and Validation	13
9.4	Benefits in a Secure Context	13

10 Comprehensive Audit Logging and Monitoring	13
10.1 Security Justifications for Architectural Choices	13
10.2 Cryptographic Log Integrity	14
10.3 Comprehensive Event Logging	14
10.4 Real-time Anomaly Detection	15
10.5 Machine Learning for Advanced Threat Detection	15
10.6 Security Dashboard and Monitoring	15
10.7 Compliance and Forensic Readiness	15
11 Additional Security Measures	16
11.1 Bot Mitigation and CAPTCHA Protection	16
11.1.1 Threat Model	16
11.1.2 Why hCaptcha?	16
11.1.3 Layered Defense Approach	16
11.1.4 Development Mode Configuration	16
11.1.5 Server-Side Verification	17
11.1.6 Security Positioning	17
11.1.7 Checklist Coverage	17
11.2 Data Encryption	17
11.3 Authentication and Authorization	17
12 Missing Functionality: Doctor-Proposed Changes	18
12.1 Proposed Implementation Design	18
13 Conclusion	18

Abstract

This report details the design and implementation of MediVault, a secure client-server system for managing medical records. The system implements a zero-trust architecture where the server is considered untrusted, requiring end-to-end encryption and client-side security measures. This document outlines the security choices made regarding the zero knowledge proof authentication for patients and doctors, multi-factor authentication implementation, data protection mechanisms and logging system.

1 Introduction

1.1 Project Context

The project aims to implement a secure medical records management system as specified in the course requirements document. The core challenge lies in designing a system where the server cannot be trusted with sensitive information, necessitating sophisticated cryptographic protections and secure protocols.

1.2 System Overview

MediVault follows a client-server architecture with two types of users: patients and doctors. Patients own their medical records and can appoint doctors to access them. Doctors are considered trusted users who require strong authentication mechanisms. The system ensures that:

- Medical records remain confidential and tamper-proof
- Only authorized users can access sensitive data
- All communications are secured against eavesdropping and tampering
- The server learns minimal information about the data it stores

1.3 Security Philosophy

Our design follows the principle of *defense in depth* with multiple security layers. We prioritize:

1. **Confidentiality:** Data is encrypted before leaving the client device
2. **Integrity:** Cryptographic mechanisms ensure data cannot be modified undetectably
3. **Authenticity:** Strong authentication prevents impersonation attacks
4. **Non-repudiation:** Actions cannot be denied by those who performed them
5. **Forward secrecy:** Compromised keys do not endanger past communications

2 Threat Model

We assume the following threat model:

- The server is untrusted and may be compromised
- Network communications are subject to interception and manipulation
- Stored data on the server is accessible to attackers
- Users may have malicious intentions or compromised devices
- Doctors require higher trust levels than patients

3 Patient Authentication and Device Binding

3.1 Motivation and Security Requirements

Patients are considered non-trusted users who nonetheless own highly sensitive data. Consequently, patient authentication must satisfy the following requirements:

- The server must never learn or store any reusable authentication secret.
- Authentication must be resistant to phishing, replay attacks, and credential stuffing.
- Patients must be able to authenticate from multiple devices.
- The solution must remain usable for non-technical users.

To meet these constraints, we selected **WebAuthn** as the primary authentication mechanism for patients.

3.2 Why WebAuthn?

WebAuthn is a W3C standard based on public-key cryptography that replaces passwords with device-generated asymmetric key pairs. Its security properties are particularly aligned with the project constraints:

- **Zero-Knowledge Authentication:** The server never receives any secret material. Authentication is performed using cryptographic challenges and signatures.
- **Phishing Resistance:** Authentication keys are bound to the origin (domain), preventing credential reuse on malicious websites.
- **No Shared Secrets:** Unlike passwords, no secret can be replayed or leaked through server compromise.
- **Hardware-Backed Security:** Keys can be protected by secure enclaves, TPMs or hardware security modules available on modern devices.

These properties make WebAuthn strictly superior to password-based authentication in

a zero-trust server model.

3.3 WebAuthn Implementation Details

Our implementation leverages the full WebAuthn specification as supported by modern browsers:

- **Physical Security Keys:** Devices like Yubikey with fingerprint sensors provide hardware-backed authentication.
- **Platform Authenticators:** Smartphones (Android with Google Password Manager) can serve as secure authenticators through passkey technology.
- **Cross-Device Authentication:** Patients can authenticate using QR codes scanned with their mobile devices, which then perform WebAuthn authentication locally.

This implementation is not merely a QR-code login flow but a complete WebAuthn integration that supports the full range of modern authentication devices.

3.4 UMK and PMK Design with PRF Support

Our implementation distinguishes two cryptographic key levels with proper pseudo-random function (PRF) support:

3.4.1 Primary Master Key (PMK)

The PMK is generated during the initial WebAuthn registration process and is bound to the authenticator (device). It is stored exclusively within secure hardware boundaries.

Security rationale:

- The PMK provides a stable cryptographic identity for the patient.
- It enables strong device binding without relying on server-side secrets.
- Loss or compromise of the server does not compromise patient authentication.

3.4.2 User Master Key (UMK) Generation and Storage

The UMK is randomly generated and then encrypted using a PRF derived from the PMK within the security key/device. This design requires devices that support PRFs, such as:

- Modern security keys (Yubico)
- Android devices with Google Password Manager
- Other WebAuthn-compliant devices with PRF capability

The encrypted UMK is stored on the server, with the following security properties:

- **Single UMK per Patient:** Each patient has one UMK regardless of the device used.

- **Multiple Encrypted Versions:** The UMK has an encrypted version for each of the patient’s registered security keys/devices.
- **Cross-Device Accessibility:** Patients can decrypt their UMK on any registered device using their PMK and the appropriate PRF.
- **Zero Server Knowledge:** The server never sees the plaintext UMK, only its encrypted versions.

This separation ensures that authentication compromise does not automatically imply data decryption capability.

4 End-to-End Encryption Scheme

4.1 File Encryption Architecture

Our system implements a sophisticated end-to-end encryption scheme where each medical file is protected with individual encryption keys:

- **File Encryption Key (FEK):** Each file has a unique FEK generated using cryptographically secure random number generation.
- **FEK Storage:** The FEK is encrypted with the patient’s UMK and stored alongside the encrypted file on the server.
- **Dynamic Key Management:** With each file modification, a new FEK is generated, ensuring forward secrecy for updated content.

4.2 Doctor Access and Key Sharing

To enable appointed doctors to access patient files, we implement a secure key distribution mechanism:

- **Doctor Public Key Encryption:** For each appointed doctor, the FEK is encrypted with the doctor’s public key (RSA-OAEP with SHA-256).
- **Secure Storage:** These encrypted FEK versions are stored on the server alongside the encrypted file.
- **Access Workflow:** When a doctor needs to access a file:
 1. Retrieve the FEK encrypted with their public key
 2. Decrypt the FEK using their private key
 3. Use the FEK to decrypt the medical file

4.3 Access Revocation Mechanism

Our design provides robust access revocation through cryptographic key rotation:

- **Key Rotation on Modification:** Each time a patient modifies a file, a new FEK is generated.
- **Automatic Revocation:** Doctors who have been revoked by the patient will not receive the new FEK encrypted with their public key.
- **Historical Access Prevention:** While revoked doctors retain access to old versions of files (encrypted with previous FEKs), they cannot access newly modified content or newly created files.
- **Granular Control:** Patients can revoke access at the individual file level or for all files simultaneously.

This approach ensures that access control is cryptographically enforced rather than relying on server-side permission checks.

5 Doctor Authentication and Key Management

5.1 Certificate-Based Authentication

Doctors authenticate using a two-factor approach:

- **Primary Factor:** X.509 client certificates issued by our PKI (detailed in Section 5)
- **Secondary Factor:** Password-based authentication

While the certificate alone provides strong authentication (effectively 2FA through possession of private key), we add password verification for defense in depth.

5.2 Password-Based Private Key Protection

The doctor's private key is stored securely using password-derived encryption:

- **Client-Side Key Generation:** Doctors generate their RSA key pairs locally during registration.
- **Password-Based Encryption:** The private key is encrypted using AES-256-GCM with a key derived from the doctor's password using PBKDF2 with SHA-256 and appropriate iteration count.
- **Secure Storage:** The encrypted private key is stored on the server, but the password is never transmitted or stored server-side.
- **Decryption Workflow:** During authentication:
 1. Doctor provides their password
 2. Encrypted private key is retrieved from server

3. Password-derived key decrypts the private key locally
4. Decrypted private key is used for certificate authentication

5.3 WebAuthn Enhancement (Future Implementation)

For additional security, WebAuthn could be implemented for doctors as a third authentication factor:

- **Multi-Factor Authentication:** Certificate + Password + WebAuthn would provide three-factor authentication.
- **Enhanced Security:** This would protect against password compromise and certificate theft simultaneously.
- **Usability Consideration:** The current two-factor approach balances security and usability for medical professionals.

6 PKI Implementation and Security Choices

6.1 Hierarchical Certificate Authority Structure

We implemented a three-tier PKI hierarchy as shown in Figure 1. This structure was chosen for several security reasons:

```

Root CA (offline, 10 years)
  --- Intermediate CA (2 years)
    --- Doctor Certificates (24 hours)

```

Figure 1: PKI Certificate Hierarchy

Why a 3-tier hierarchy?

- **Root CA Isolation:** The Root CA is kept offline (air-gapped), making it immune to network-based attacks. Compromise of this key would require physical access, significantly reducing the attack surface.
- **Operational Separation:** The Intermediate CA handles daily certificate issuance. If compromised, it can be revoked without affecting the entire PKI infrastructure, allowing for graceful recovery.
- **Separation of Duties:** Different keys serve different purposes, following the principle of least privilege. The Root CA only signs Intermediate CA certificates, never end-entity certificates.
- **Scalability:** The hierarchy allows for future expansion (e.g., separate Intermediate CAs for different hospitals or regions) without rebuilding the entire trust infrastructure.

Why 24-hour doctor certificates?

- **Elimination of Revocation Infrastructure:** Short-lived certificates (24-hour validity) eliminate the need for Certificate Revocation Lists (CRLs) or Online Certificate Status Protocol (OCSP), which are complex and often unreliable in practice.
- **Reduced Exposure Window:** If a doctor's private key is compromised, the maximum damage window is limited to 24 hours, after which the certificate automatically expires.
- **Forced Re-authentication:** Daily certificate renewal ensures doctors regularly re-authenticate, increasing accountability and reducing the risk of unauthorized long-term access.
- **Modern Zero-Trust Practice:** This approach follows Google's BeyondCorp model, where certificates are treated as ephemeral credentials rather than long-term identities.

Cryptographic Algorithm Choices:

- **ECDSA P-384 with SHA-256:** Selected for its 192-bit security level (equivalent to RSA-7680), offering strong security with smaller key sizes and faster operations compared to RSA. SHA-256 is used as the hash function, providing state-of-the-art cryptographic security.
- **CSR-Based Enrollment:** Doctors generate Certificate Signing Requests (CSRs) locally, ensuring their private keys never leave their devices. This provides true non-repudiation since only the doctor possesses the private key.
- **PKCS#12 Format:** Certificates are distributed in PKCS#12 bundles with strong passwords for browser import, ensuring secure storage on client devices.

6.2 Certificate Lifecycle Management

The certificate issuance process follows these security principles:

1. **Offline Root CA:** The Root CA key is generated on an air-gapped machine and stored on encrypted media. It is only brought online for Intermediate CA certificate signing.
2. **Automated Intermediate CA:** The Intermediate CA runs on a secured server with strict access controls. It automatically issues doctor certificates with 24-hour validity.
3. **Client-Side Key Generation:** Doctors generate key pairs locally, submitting only the public key (in CSR form) to the CA. Private keys remain exclusively on doctor devices.
4. **Audit Trail:** All certificate issuances are logged with timestamps, requester identity, and certificate serial numbers for forensic analysis.

7 Server Trust Configuration

The server is configured to:

- Trust only the Intermediate CA certificate (not individual doctor certificates)
- Require client certificates for all `/doctor/**` endpoints
- Validate complete certificate chains (Doctor $\rightarrow$ Intermediate $\rightarrow$ Root)
- Extract doctor identity from certificate Subject fields (Common Name and Organization)
- Enforce certificate validity periods (automatic rejection of expired certificates)

This configuration ensures that the server can authenticate doctors without needing to store sensitive authentication material, aligning with the requirement that the server should not be trusted with such information.

8 Secure Input Processing and Validation

8.1 Context-Specific Security Requirements

MediVault handles two user types with different trust levels:

- **Patients:** Provide personal and medical data
- **Doctors:** Authenticated via certificates for record access

Sensitive data includes:

- Patient identity information
- Client-side encrypted medical data
- File metadata
- WebAuthn authentication information

8.2 Input Validation Strategy

We implement a multi-layered validation approach:

- **Dedicated DTOs per Endpoint:** Each API endpoint accepts only specific Data Transfer Objects, limiting accepted fields to prevent mass-assignment attacks and injection attempts.
- **Bean Validation:** Automatic validation of field presence, format, and length using Java Bean Validation annotations. This prevents malformed or malicious inputs from reaching business logic.

- **JSON Error Responses:** Validation errors are returned in structured JSON format for frontend processing, preventing information leakage while maintaining usability.

This approach prevents:

- Data injection attacks
- Mass-assignment vulnerabilities
- Malformed or malicious inputs

8.3 XSS Protection

- **Client-Side Escaping:** All user data is properly escaped before DOM insertion, treating patient names, file metadata, and other user-controlled content as potentially malicious.
- **Zero-Trust Frontend Architecture:** We assume no data from the server is inherently safe, implementing defensive rendering throughout the application.

8.4 Canonicalization and Normalization

- **Unicode Normalization (NFC):** Applied to names, keys, and metadata before storage and validation to ensure consistency and prevent homoglyph attacks.
- **Consistent Processing:** Canonical forms are used consistently across the system, eliminating encoding-based bypass techniques.

8.5 Denial of Service Prevention

- **Size Limits:** Adapted limits for names, keys, and medical files prevent resource exhaustion attacks.
- **Strict Format Validation:** Only accepted character sets and formats are allowed, reducing parsing complexity and attack surface.

8.6 Defensive Layered Architecture

Validation occurs at multiple levels:

1. **Frontend:** JavaScript validation for user experience
2. **Transport:** Bean validation on server-side DTOs
3. **Business:** Complementary validation in service layer

This defense-in-depth approach ensures security even if one layer is bypassed.

8.7 STRIDE Threat Coverage

The secure input processing system addresses key STRIDE threats:

- **Spoofing:** Strict identity validation
- **Tampering:** Input validation and canonicalization
- **Repudiation:** Complete audit logging (complements logging system)
- **Information Disclosure:** XSS escaping
- **Denial of Service:** Size limits and validation
- **Elevation of Privilege:** WebAuthn authentication prevents privilege escalation through input manipulation

9 Software Architecture and Design Decisions

9.1 Use of Data Transfer Objects (DTOs)

In the MediVault system, we adopted the Data Transfer Object (DTO) pattern to facilitate secure and efficient data exchange between the client and server components. DTOs are simple objects that encapsulate data and are used to transfer information without exposing internal implementation details or sensitive fields.

9.1.1 Motivation for DTOs:

- **Security:** By using DTOs, we ensure that only the intended data is serialized and transmitted, reducing the risk of accidental data leakage.
- **Decoupling:** DTOs decouple the internal domain models from the API contracts, allowing us to evolve the backend logic without breaking client compatibility.
- **Validation:** DTOs provide a clear structure for input validation, ensuring that only well-formed and expected data is processed by the server.
- **Consistency:** They enforce a consistent data format across different API endpoints, simplifying client-side parsing and error handling.
- **Minimizing Attack Surface:** By explicitly defining which fields are exposed, we reduce the risk of over-posting or mass assignment vulnerabilities.

9.1.2 Example:

For instance, when a doctor requests access to a patient’s medical record, the server responds with a DTO containing only the encrypted file, the file metadata (encrypted), and the access grant status—never the raw database object or internal audit fields.

9.2 Layered Architecture

The project follows a layered architecture, separating concerns into distinct modules:

- **Controller Layer:** Handles HTTP requests, authentication, and input validation.
- **Service Layer:** Implements business logic, cryptographic operations, and access control.
- **Data Access Layer:** Manages persistence, data retrieval, and mapping between domain models and DTOs.

This separation improves maintainability, testability, and security by ensuring that each layer has a clear responsibility and limited knowledge of other layers' internals.

9.3 API Design and Validation

All API endpoints are designed to accept and return DTOs. Input data is validated against strict schemas to prevent injection attacks and malformed requests. Output DTOs are constructed explicitly, never by serializing internal objects directly.

9.4 Benefits in a Secure Context

In a zero-trust architecture, minimizing the data surface and controlling serialization is critical. DTOs help enforce these boundaries, making it easier to audit, test, and reason about the data flows in the system.

10 Comprehensive Audit Logging and Monitoring

10.1 Security Justifications for Architectural Choices

Why Separate Logging Infrastructure?

- **Failure Isolation:** If the application server crashes or is compromised, audit logs remain intact and uncorrupted.
- **Security Boundary:** Audit logs cannot be tampered with by compromised application code since they're written to a separate LogServer via TCP.
- **Performance:** Asynchronous TCP logging doesn't block application request processing, maintaining system responsiveness.
- **Scalability:** The LogServer can handle multiple applications, supporting future expansion.

Why Elastic Stack for Log Management?

- **Immutability:** Elasticsearch indices are append-only, preventing log tampering.
- **Real-time Threat Detection:** Kibana alerts provide instant notification of secu-

rity incidents.

- **Machine Learning Anomaly Detection:** Automated detection of unusual patterns.
- **Full-text Search:** Rapid forensic investigation across millions of log entries.
- **Regulatory Compliance:** Comprehensive audit trails satisfy healthcare compliance requirements.

Why Filebeat as Log Shipper?

- **Persistent Buffer:** The audit.log file acts as a buffer, preventing log loss during Elasticsearch outages.
- **Automatic Retries:** Filebeat handles delivery retries automatically, ensuring log delivery.
- **Centralized Parsing:** Log parsing logic is centralized in Filebeat configuration, ensuring consistency.
- **Replay Capability:** Logs can be replayed if Elasticsearch data is lost or corrupted.

10.2 Cryptographic Log Integrity

We implement blockchain-style hash chaining for log integrity:

`timestamp|severity|ip|actor|action|target|extra_data|previous_hash|hash`

Security Benefits:

- **Tamper Evidence:** Each log entry includes the hash of the previous entry (HMAC-SHA256), creating an immutable chain.
- **Detectable Modifications:** Any alteration to historical logs breaks the hash chain.
- **Non-repudiation:** Cryptographic hashing ensures actions cannot be denied.

10.3 Comprehensive Event Logging

Security-critical events are meticulously logged:

- **Authentication Events:** All login attempts (success/failure) with IP addresses
- **Access Control:** File accesses, modifications, and permission changes
- **Certificate Operations:** Issuance, renewal, and validation of doctor certificates
- **Patient Consent Actions:** Granting and revoking doctor access
- **System Changes:** Configuration modifications and administrative actions

10.4 Real-time Anomaly Detection

We configured eight security anomaly detectors in Kibana:

Table 1: Security Anomaly Detection Rules

#	Name	Priority	Detection Criteria
1	Brute Force	Critical	>5 failed logins in 5 minutes
2	Hash Chain Integrity	Critical	Blockchain validation failure
3	Unauthorized Admin	Critical	Admin actions by non-admins
4	Mass Data Access	High	>20 downloads in 10 minutes
5	After-Hours Access	High	Access between 22:00-06:00
6	Unusual IP per User	Medium	Rare IP addresses (ML-based)
7	Patient Record Deletion	Medium	DELETE actions on patient records
8	Rare Actions	Medium	Unusual actions globally (ML-based)

10.5 Machine Learning for Advanced Threat Detection

We leverage Elasticsearch’s ML capabilities for:

- **Behavioral Baselines:** Establishing normal patterns per user and system
- **Anomaly Scoring:** Quantitative risk assessment of suspicious activities
- **Adaptive Detection:** Continuous learning from new log data

10.6 Security Dashboard and Monitoring

A dedicated Kibana security dashboard provides:

- Real-time visualization of security events
- Geographic mapping of access attempts
- User activity heatmaps
- Automated alerting for critical incidents

10.7 Compliance and Forensic Readiness

The logging system supports:

- **Regulatory Compliance:** Meets healthcare data protection requirements
- **Forensic Investigations:** Complete audit trail for incident response
- **Non-repudiation:** Cryptographic proof of actions
- **Accountability:** Clear attribution of actions to specific users

11 Additional Security Measures

11.1 Bot Mitigation and CAPTCHA Protection

11.1.1 Threat Model

Authentication endpoints are prime targets for:

- brute-force attacks,
- automated account creation,
- credential stuffing,
- denial-of-service amplification.

While cryptographic authentication mechanisms such as WebAuthn and PKI provide strong identity guarantees, they do not inherently protect against large-scale automated abuse. To mitigate this risk, we integrated **hCaptcha** as a pre-authentication filter.

11.1.2 Why hCaptcha?

hCaptcha was selected for the following reasons:

- It effectively distinguishes humans from automated scripts.
- It integrates seamlessly with modern web applications.
- It does not require storing additional sensitive information.
- It aligns with OWASP recommendations for authentication hardening.

11.1.3 Layered Defense Approach

The CAPTCHA mechanism is implemented as a two-layer defense:

- **Invisible CAPTCHA:** Runs silently and analyzes behavioral signals. Legitimate users experience no friction.
- **Visible CAPTCHA (Fallback):** Triggered when confidence is low or when explicitly enabled for testing purposes.

This design minimizes usability impact while maintaining strong protection.

11.1.4 Development Mode Configuration

For demonstration and testing purposes, we intentionally use hCaptcha **development keys** with the *Always Challenge* option enabled.

Justification:

- Allows reviewers to systematically observe CAPTCHA behavior.

- Avoids exposure of production credentials.
- Facilitates security testing and validation.

11.1.5 Server-Side Verification

All CAPTCHA tokens are verified server-side using the hCaptcha verification API. Client-side validation alone is never trusted.

This prevents:

- token replay,
- client-side bypass,
- forged CAPTCHA responses.

11.1.6 Security Positioning

hCaptcha is not used as an authentication mechanism but as a complementary protection layer. It reduces attack surface without replacing cryptographic identity verification.

11.1.7 Checklist Coverage

This mechanism addresses:

- Authentication hardening (Checklist #2)
- Automated attack mitigation
- Monitoring and abuse prevention (Checklist #10)

11.2 Data Encryption

- **End-to-End Encryption:** Medical records are encrypted with patient-controlled keys before upload to the server. The server never sees plaintext medical data.
- **Key Management:** Each patient has a master key pair. File encryption keys are derived using HKDF and encrypted with the patient's public key for sharing with appointed doctors.
- **Metadata Protection:** Following master's requirements, metadata (timestamps, sizes, privileges) is sent separately and cannot be correlated with encrypted content through encryption with different keys.

11.3 Authentication and Authorization

- **Multi-factor Authentication:** Doctors use certificate-based authentication combined with password verification.
- **Fine-grained Access Control:** Patients control exactly which doctors can access which files, with the ability to grant and revoke access dynamically.

- **Consent Mechanism:** When doctors initiate actions (uploading files, accessing records), explicit patient approval is required through a separate secure channel.

12 Missing Functionality: Doctor-Proposed Changes

The current system allows doctors to view patient medical records but lacks the capability for doctors to propose changes or additions to those records. This was a planned feature that would have completed the doctor-patient collaboration workflow.

12.1 Proposed Implementation Design

Had time permitted, we would have implemented doctor proposed changes using the following cryptographic design:

- **Patient RSA Key Pair:** Patients would generate an RSA key pair during registration, storing the public key on the server and keeping their private key encrypted with AES-GCM using their User Master Key (UMK).
- **Doctor File Proposal Process:**
 1. Doctor generates a File Encryption Key (FEK)
 2. Encrypts the file and filename using AES-GCM with the FEK
 3. Encrypts the FEK using the patient's public key (RSA-OAEP with SHA-256)
 4. Stores both encrypted components on the server
- **Patient Review and Approval:**
 1. Patient downloads and decrypts the proposed change using their private RSA key
 2. Reviews the content
 3. If approved, generates a new FEK and re-encrypts the file
 4. Encrypts the new FEK with their UMK for personal access
 5. Encrypts the FEK with each appointed doctor's public key for shared access

13 Conclusion

The MediVault system successfully demonstrates that secure medical records management is achievable in a zero trust environment where the server cannot be trusted. Our implementation combines cryptographic protections with defense-in-depth security measures to protect sensitive healthcare data while maintaining usability.

Key achievements include zero-knowledge WebAuthn authentication for patients with proper PRF-based UMK encryption, a three-tier PKI with 24-hour doctor certificates,

and true end-to-end encryption with patient-controlled keys and cryptographic access revocation. The architecture ensures medical data remains confidential and tamper-proof throughout all its lifecycle.

Security monitoring is provided through immutable audit logging with blockchain-style hash chaining, real-time anomaly detection, and machine learning for identifying subtle attack patterns. Secure input processing, multilayered validation, and CAPTCHA integration provide comprehensive protection against common threats.

MediVault successfully implements our security philosophy of confidentiality, integrity, authenticity, non-repudiation, and forward secrecy. The system proves that with careful cryptographic design and layered security controls, sensitive medical data can be protected even when storing it on untrusted infrastructure.